IMPERIAL COLLEGE LONDON

DEPARTMENT OF ELECTRICAL AND ELECTRONIC ENGINEERING
EXAMINATIONS

DISCRETE MATHEMATICS

Tuesday, 12 May 2026, 2:30 PM

Time allowed: 2:00 hours

There are THREE questions on this paper.

Answer ALL questions.

Any special instructions for invigilators and information for candidates are on page 1.

Examiners responsible First Marker(s) : D. Goodman
Second Marker(s) : Sen Wang

© Imperial College London

DISCRETE MATHEMATICS

1. This question carries 30% of the marks.

   For each of the following statements, state whether it is true or false and provide a supporting proof.

   a) $3n - 5 = \Theta(n)$. [3]

   b) $n^2 + 7n + 3 = O(n)$. [3]

   c) $2^n = \Omega(n^3)$. [3]

   d) $n^2 + n = \Theta(n \log n)$. [3]

   e) $\log_2 n = O(\log_3 n)$. [3]

2. This question carries 20% of the marks.

   Consider the following pseudocode for computing the maximum value of an array $A[1 \ldots n]$ of integers.

```
function MAX(A)
    n ← length(A)
    m ← A[1]
    for i ← 2 to n do
        if A[i] > m then
            m ← A[i]
        end if
    end for
    return m
end function
```

   a) State a loop invariant for the `for` loop.

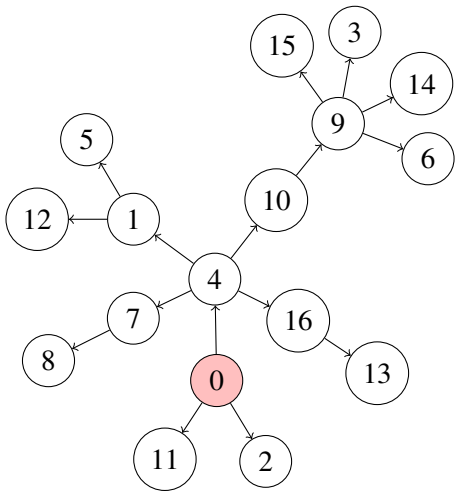
   Recall: a loop invariant is a property that holds at the start of each iteration of the loop and is preserved through each iteration, and that is useful for proving the correctness of the algorithm. [3]

   b) Prove initialization and maintenance of your invariant. [5]

   c) Use the invariant to prove the algorithm returns the maximum value in $A[1 \ldots n]$. [2]

3. This question carries 50% of the marks.

Consider the following problem for scheduling a list of n tasks to run on m processors. Each task takes a unit of time to run. Each processor can run at most one task at a time. Some tasks cannot start until other tasks have finished. We define a graph $G = (V, E)$ with nodes V corresponding to tasks, and a directed edge $(i, j) \in E$ from node i to node j if task i depends on task j (i.e., j must finish before i can start). The problem is to find a schedule for all tasks that respects the dependencies and minimizes the total time to complete all tasks. An example of such a graph is shown below, where task 0 (in pink) is the final task to be completed, which depends on tasks 2, 4 and 11, and so on.



Hu's algorithm gives an optimal solution for this problem when the graph is a tree with a single root node, as in the example above (0 is the root node). First, we define the *level* of each node to be the shortest distance from the root node. In the example above the root node 0 has level 0, nodes 2, 4 and 11 have level 1, and so on. Hu's algorithm says that whenever a processor becomes available, we should schedule any task that satisfies the conditions (C_1) it has not yet been started, (C_2) all its dependencies have completed, and (C_3) it has the highest level among all tasks satisfying C_1 and C_2.

a) Apply a breadth-first search to compute the level of each node in the example graph above.

   i) List all the nodes at each level. Give your answer in the form of a table with one row for each level, and list all the nodes in that level in increasing order. [5]

   ii) In the case of an infinite number of processors ($m = \infty$), what is the minimum time to complete all tasks and why? [2]

b) Apply Hu's algorithm to the example graph in the case $m = 3$. Use the extra condition (C_4) that we pick the task with the lowest index among all tasks satisfying $C_1 - C_3$. List the tasks scheduled at each time step for each processor in a table, and give the total time to complete all tasks. [3]

c) We want to prove that Hu's algorithm always gives an optimal schedule. The first step is to compute a lower bound on the time to complete all tasks that any schedule must satisfy. To find this lower bound, let V_L be the set of nodes whose level is greater than or equal to L. Let h be the maximum level of any node.

   i) If we could ignore all dependencies between tasks in V_L and just schedule them as if they were independent, but using only m processors, how many time steps would it take to complete all tasks in V_L? [2]

ii) For the remaining tasks in $V - V_L$, how many time steps would it take to complete all those tasks if we had an infinite number of processors? [2]

iii) Using your answers to the previous two parts, and considering all possible values of L, give a lower bound on the time to complete all tasks that any schedule must satisfy. [1]

iv) Compute this lower bound for the example graph above, and show that it matches the time taken by Hu's algorithm. [1]

d) We will complete this proof by showing this lower bound is attained.

i) Let S_t be the set of tasks scheduled by Hu's algorithm at time step t. Let A be the set of tasks that can be scheduled at a given moment (i.e. they satisfy C_1 and C_2). Show that $|S_0| \geq |S_1| \geq |S_2| \geq \ldots$.

Hint: consider what can happen to $|A|$ each time a scheduling operation takes place in Hu's algorithm. [2]

ii) Show that from one time step to the next, the highest level among all remaining tasks cannot decrease by more than 1. [2]

iii) Let k be the last time step at which Hu's algorithm schedules precisely m tasks all from the same level, and let L be that level. Let L' be the highest level among all tasks remaining after time step k. Show that the schedule takes precisely $L' + 1$ steps after stage k. [2]

iv) Show that for all stages up to and including stage k, all processors are fully utilised. [1]

v) Show that the schedule takes precisely $\lceil |V_L|/m \rceil + L$ steps, demonstrating that Hu's algorithm achieves the lower bound.

Hint: consider the two cases where S_k uses up all tasks at level L, and where it does not. [2]